

Sistema Integral de Gestión de Eventos

Arquitectura de Software

Atributos de Calidad y Propuesta Arquitectónica

Portal Web Cliente

Portal Web Administrativo / Operativo

Aplicaciones Móviles: Cliente y Personal

Materia: Arquitectura de Software

Estudiante: Samuel Contreras

Fecha: Agosto de 2026

Índice

1	Introducción	2
2	Contexto y alcance	2
3	Interfaces del sistema	3
3.1	Portal Web del Cliente	3
3.2	Portal Web Administrativo y Operativo	3
3.3	Aplicación Móvil del Cliente	3
3.4	Aplicación Móvil del Personal	4
4	Atributos de calidad priorizados	4
5	Arquitectura propuesta	5
5.1	API Gateway y balanceador	6
5.2	Microservicios	7
6	Infraestructura de soporte	7
6.1	Redis	7
6.2	RabbitMQ / Kafka	7
6.3	Bases de datos	7
7	Servicios externos	8
7.1	Pasarela de pagos	8
7.2	Servicio de notificaciones	8
8	Cómo la arquitectura satisface los atributos de calidad	8
8.1	Consistencia – Alta	8
8.2	Disponibilidad – Alta	8
8.3	Desplegabilidad – Media	11
8.4	Integrabilidad – Media	13
8.5	Comprobabilidad (<i>Testability</i>) – Media	15
8.6	Seguridad física (<i>Safety</i>) – Baja	16
8.7	Seguridad – Alta	17
8.8	Rendimiento – Alta	20
8.9	Tiempo real – Media-Alta	21
8.10	Escalabilidad – Media	21
8.11	Trazabilidad – Media	22
8.12	Usabilidad – Baja-Media	22
8.13	Mantenibilidad – Baja-Media	23

8.14	Portabilidad – Baja-Media	24
9	Caso complejo: Gestión de Evacuación ante Emergencias	24
9.1	Objetivo	25
9.2	Actores	25
9.3	Flujo principal	25
9.4	Flujos alternativos	26
9.5	Excepciones	26
9.6	Atributos de calidad del caso	26
10	Comunicación durante una emergencia	26
11	Relación entre interfaces y microservicios	27
12	Resumen de decisiones arquitectónicas	28
13	Conclusión	28

1. Introducción

Este documento presenta una segunda versión del análisis de atributos de calidad y de la propuesta de arquitectura para el **Sistema Integral de Gestión de Eventos**. La propuesta toma como base la primera versión del proyecto, donde se planteó una organización mediana-grande que gestiona eventos masivos, como conciertos, festivales y partidos deportivos, con miles de usuarios concurrentes, especialmente durante la apertura de venta y el ingreso al recinto.

El sistema centraliza procesos de boletería, mercado secundario de entradas, administración de personal, parqueaderos y operación de eventos. En esta segunda versión se define con mayor precisión la distribución de funcionalidades entre las interfaces.

Se contemplan cuatro clientes principales:

- **Portal Web del Cliente:** compra, consulta y gestión de entradas, reventas, promociones y parqueaderos.
- **Portal Web Administrativo y Operativo:** utilizado por administradores, organizadores y trabajadores autorizados.
- **Aplicación Móvil del Cliente:** principalmente para visualizar entradas y códigos QR, información y notificaciones.
- **Aplicación Móvil del Personal:** para turnos, asistencia, instrucciones, reportes y alertas.

La primera propuesta priorizó consistencia, disponibilidad, seguridad y rendimiento. Esta versión mantiene esos atributos y aumenta la relevancia de tiempo real, trazabilidad, escalabilidad, usabilidad y portabilidad.

2. Contexto y alcance

El sistema está orientado a clientes, organizadores, administradores, supervisores, personal operativo y proveedores. Los principales dominios son:

1. Venta y gestión de boletería.
2. Mercado secundario de entradas.
3. Gestión de parqueaderos.
4. Logística y operación del evento.
5. Administración general.

6. Gestión del personal.
7. Gestión de emergencias y evacuaciones.

La arquitectura debe permitir que estos dominios evolucionen de forma relativamente independiente y que los componentes con alta demanda puedan escalar sin afectar innecesariamente a los demás.

3. Interfaces del sistema

3.1. Portal Web del Cliente

El portal web permite registro e inicio de sesión, consulta de eventos, compra de entradas, consulta de entradas adquiridas, publicación y compra de entradas revendidas, cancelaciones y devoluciones, promociones, reserva de parqueadero y recepción de notificaciones.

La lógica de negocio crítica permanece en el backend y el portal consume los servicios mediante el API Gateway.

3.2. Portal Web Administrativo y Operativo

Está dirigido a administradores, organizadores y trabajadores autorizados. Incluye:

- CRUD de eventos, usuarios, clientes, empleados y roles.
- CRUD de localidades y tipos de entrada.
- CRUD de parqueaderos y zonas.
- CRUD de proveedores, recursos logísticos y promociones.
- Planificación de eventos.
- Control de aforo.
- Gestión de incidentes.
- Gestión de emergencias y evacuaciones.
- Monitoreo del evento.

3.3. Aplicación Móvil del Cliente

Permite iniciar sesión, consultar eventos y entradas, visualizar el código QR de cada entrada, consultar información del evento y recibir notificaciones y alertas. Su función crítica es facilitar el acceso rápido a los QR durante el ingreso.

3.4. Aplicación Móvil del Personal

Permite consultar eventos asignados, turnos, horarios y áreas de trabajo; confirmar asignaciones; solicitar cambios de turno; registrar ingreso y salida; recibir instrucciones; reportar incidentes y recibir alertas de emergencia.

4. Atributos de calidad priorizados

Prioridad	Atributo	Justificación
Alta	Consistencia	Una entrada no puede pertenecer a dos propietarios ni venderse dos veces simultáneamente. También se requiere consistencia en pagos, transferencias, asignaciones y asistencia.
Alta	Disponibilidad	El sistema debe estar disponible durante ventas, ingreso masivo, operación del evento y emergencias.
Alta	Seguridad	Deben protegerse datos personales, credenciales, pagos, QR, información de empleados y permisos.
Alta	Rendimiento	Miles de usuarios pueden realizar operaciones concurrentes y las validaciones deben responder rápidamente.
Media-Alta	Tiempo real	Aforo, entradas, estado del personal, parqueaderos y emergencias deben actualizarse casi inmediatamente.
Media	Escalabilidad	Debe ser posible aumentar las instancias de los servicios con mayor demanda.
Media	Trazabilidad	Deben auditarse transferencias, pagos, accesos, turnos, operaciones administrativas y emergencias.
Baja-Media	Usabilidad	Las interfaces deben adaptarse a clientes, empleados, supervisores, organizadores y administradores.
Baja-Media	Mantenibilidad	Los cambios en un dominio no deberían afectar innecesariamente a los demás.
Baja-Media	Portabilidad	Las funcionalidades deben poder utilizarse mediante web y aplicaciones móviles.

Media	Integrabilidad	El sistema depende de dos integraciones externas críticas para el negocio (pasarela de pagos, proveedor de notificaciones) que deben poder cambiarse o fallar sin comprometer el flujo principal.
Media	Desplegabilidad	Los microservicios deben poder actualizarse (hot-fixes, nuevas versiones) sin interrumpir un evento en curso, especialmente durante ingreso masivo o apertura de venta.
Media	Comprobabilidad	Los flujos críticos (reventa concurrente, pagos, protocolo de emergencia) deben poder probarse de forma aislada, sin depender de servicios externos reales.
Baja	Seguridad física (<i>Safety</i>)	El sistema no controla hardware crítico, pero sí participa en la seguridad física de los asistentes a través del control de aforo y la coordinación de evacuaciones.

Las cuatro filas añadidas (Integrabilidad, Desplegabilidad, Comprobabilidad y Seguridad física) corresponden a atributos vistos en el curso (Clases 7, 10, 11 y 13) que no estaban priorizados en la versión inicial de esta tabla; se incorporan aquí para que las tácticas y patrones aplicados en la Sección 8 tengan una prioridad explícita de la que derivarse, tal como exige el curso para todo el rango de clases 6 a 14.

5. Arquitectura propuesta

Se propone una arquitectura distribuida basada en **microservicios**, con API Gateway, balanceador de carga, Redis, RabbitMQ o Kafka y bases de datos independientes por dominio.

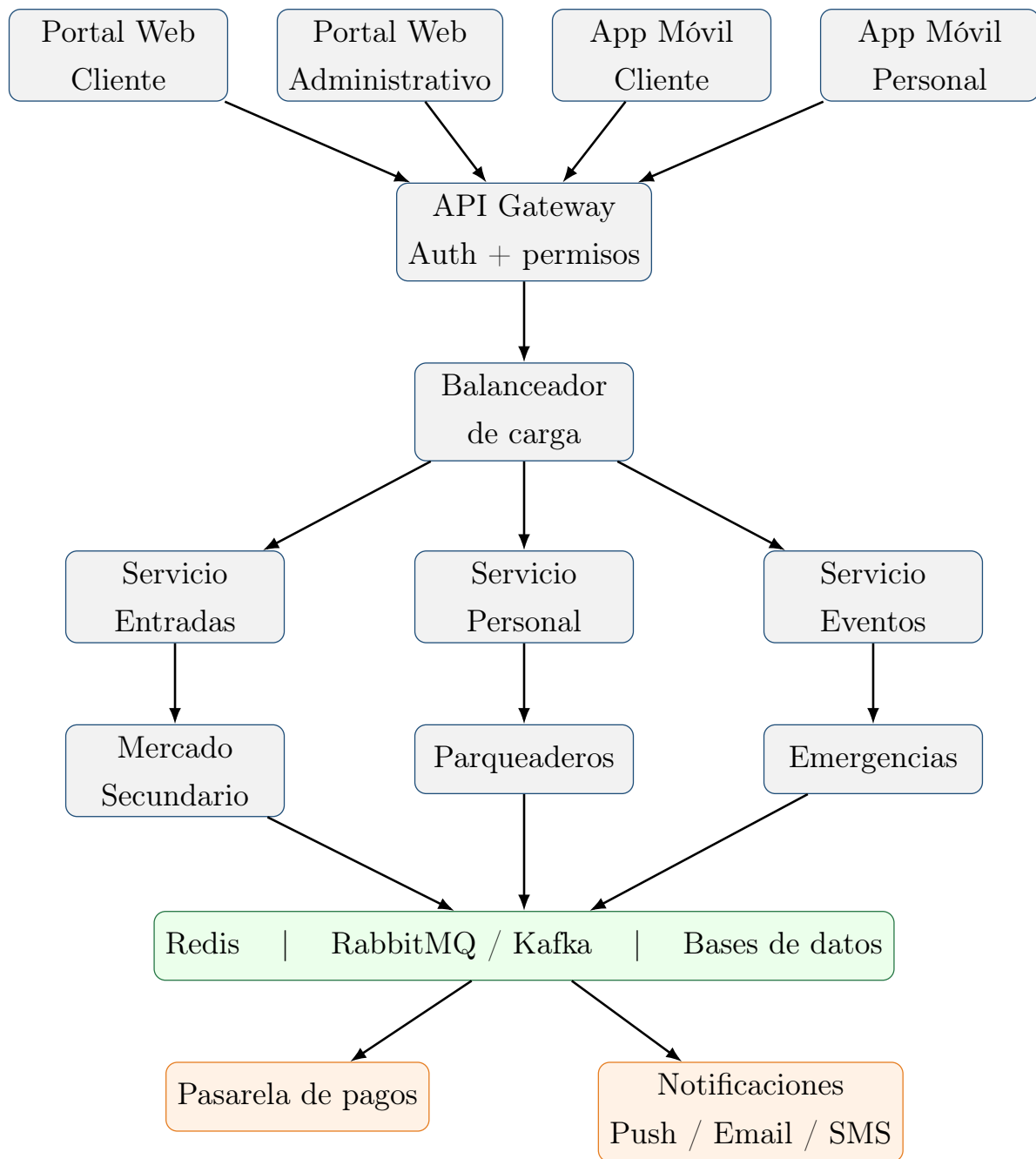


Figura 1: Arquitectura general propuesta.

5.1. API Gateway y balanceador

El API Gateway es el punto de entrada al backend. Centraliza autenticación, autorización, validación de tokens y enrutamiento. El balanceador distribuye solicitudes entre múltiples instancias y permite mantener el servicio disponible cuando una instancia falla.

5.2. Microservicios

- **Entradas:** compra, disponibilidad, validación y gestión de entradas.
- **Mercado secundario:** publicación, reventa, compra y transferencia de propiedad.
- **Personal:** empleados, roles, turnos, asignaciones y asistencia.
- **Eventos:** eventos, localidades, configuración y aforo.
- **Parqueaderos:** reservas, espacios, ingreso, salida, cobro y ocupación.
- **Emergencias:** incidentes, evacuaciones, zonas, rutas, alertas y seguimiento.

Cada microservicio puede disponer de su propia base de datos para reducir el acoplamiento.

6. Infraestructura de soporte

6.1. Redis

Redis se utiliza para información de acceso rápido y control de concurrencia. En el mercado secundario puede bloquear temporalmente una entrada mientras se procesa una compra, evitando que dos compradores completen simultáneamente la misma operación.

También puede apoyar información temporal de aforo, disponibilidad, sesiones y estados que requieran consultas rápidas.

6.2. RabbitMQ / Kafka

Las colas desacoplan operaciones asíncronas como generación de QR, notificaciones, auditoría, liquidación de pagos y propagación de alertas.

Un ejemplo es:

$$\text{ENTRADA_TRANSFERIDA} \rightarrow \text{COLA} \rightarrow \begin{cases} \text{Generar QR} \\ \text{Notificar} \\ \text{Auditar} \end{cases}$$

6.3. Bases de datos

Se plantea una base de datos independiente por dominio: entradas, personal, eventos, parqueaderos, emergencias y mercado secundario. Esto reduce el acoplamiento y permite que los servicios evolucionen de forma independiente.

7. Servicios externos

7.1. Pasarela de pagos

Se utiliza para compras, reventas, reembolsos y cobros del parqueadero. La integración debe manejar respuestas exitosas, rechazos, tiempos de espera y fallos.

7.2. Servicio de notificaciones

Envía notificaciones push, correo y SMS cuando corresponda. Ejemplos: confirmación de compra, reventa, nuevo QR, cambios de evento, turnos, asignaciones y emergencias.

8. Cómo la arquitectura satisface los atributos de calidad

Esta sección aplica, para cada atributo de calidad priorizado, el catálogo de tácticas y patrones de Bass/Kazman visto en su clase correspondiente (Clases 6 a 13; la Clase 14, Usabilidad, se trata aparte con el *Five Planes Framework* de Garrett, que es el marco que usó el profesor en esa clase en vez de tácticas arquitectónicas). Todas las tablas siguen el mismo criterio que ya se aplicó a Disponibilidad: identificar 2-3 escenarios reales del proyecto, comparar explícitamente al menos dos tácticas o patrones candidatos, y elegir uno con su justificación.

8.1. Consistencia – Alta

Redis puede utilizarse para bloqueo temporal de entradas durante el checkout. La persistencia de la operación se realiza en el servicio correspondiente, evitando que una entrada tenga simultáneamente dos operaciones de compra válidas.

8.2. Disponibilidad – Alta

El balanceador y las múltiples instancias permiten redirigir solicitudes hacia instancias sanas. Esto es especialmente importante durante ventas, ingreso masivo, validación QR y emergencias.

Siguiendo el catálogo de tácticas (detección, recuperación y prevención de defectos) y patrones (redundancia, *circuit breaker*) de disponibilidad de Bass/Kazman visto en clase, la Tabla 2 identifica las tácticas y patrones aplicables a los escenarios de disponibilidad más críticos del sistema.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Falla de una instancia de validación de QR durante el ingreso masivo	Una instancia deja de responder mientras miles de asistentes ingresan; el balanceador debe redirigir el tráfico a instancias sanas sin interrupción perceptible.	Heartbeat / monitoreo (detección); redundant spare + reconfiguración (recuperación); reinicio escalado para reintegrar la instancia (reintroducción).	Redundancia activa (<i>hot spare</i>) tras balanceador de carga.
Picos de tráfico en la apertura de venta	Miles de usuarios consultan y compran de forma simultánea; el servicio de Entradas debe absorber la carga sin caerse, degradando funciones no críticas si es necesario.	Monitoreo de carga (detección); autoescalado + reconfiguración (recuperación); sala de espera virtual diseñada como estado competente, no como error (prevención); <i>graceful degradation</i> .	Redundancia activa con escalado horizontal (<i>elastic</i>) del microservicio de Entradas.
Caída del API Gateway (punto único de entrada)	La instancia activa del Gateway falla; el tráfico debe redirigirse de inmediato a otra instancia sin exponer el fallo a las cuatro interfaces.	Heartbeat / <i>health-check</i> (detección); redundant spare + reconfiguración (recuperación).	Redundancia activa (<i>hot spare</i>) + <i>Circuit Breaker</i> hacia los microservicios lentos, para evitar fallos en cascada.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Indisponibilidad de la pasarela de pagos externa	La pasarela no responde o responde con error durante una compra o reventa; la operación no debe fallar por completo.	Detección de excepciones / <i>timeout</i> ; <i>retry</i> con máximo de reintentos (recuperación); <i>graceful degradation</i> (marcar la transacción como pendiente en vez de fallar toda la operación).	<i>Circuit Breaker</i> (evita seguir enviando solicitudes a una pasarela caída).
Falla del proveedor de notificaciones durante una emergencia	El proveedor de notificaciones no responde mientras se coordina una evacuación; el protocolo de evacuación no debe detenerse por esta falla.	Detección de excepciones / <i>timeout</i> ; <i>retry</i> limitado + reconfiguración hacia un canal secundario, SMS si falla el push (recuperación); <i>graceful degradation</i> (el personal en sitio releva la comunicación).	<i>Circuit Breaker</i> + redundancia pasiva (canal secundario en espera).
Falla del nodo de Redis usado para el bloqueo de concurrencia en reventa	El nodo Redis activo se cae mientras se procesa una reventa; el sistema debe conmutar a una réplica sin permitir que dos compradores completen la misma compra.	Heartbeat / <i>health-check</i> del clúster (detección); <i>redundant spare</i> (recuperación); resincronización de estado tras el <i>failover</i> (reintroducción).	Redundancia activa/pasiva vía réplica de Redis (Redis Sentinel/Cluster).

Tabla 2: Tácticas y patrones de disponibilidad aplicados a los escenarios críticos del sistema.

En resumen, la **detección** de fallas se apoya en heartbeat/health-check, monitoreo de

carga y detección de excepciones/timeout; la **recuperación** combina redundant spare, reconfiguración, reintentos (retry), degradación controlada (graceful degradation), reinicio escalado y resincronización de estado; y la **prevención** se refleja en diseñar de entrada estados competentes para los picos de tráfico esperados (sala de espera virtual) en vez de tratarlos como una excepción. A nivel de patrones, la **redundancia activa (hot spare)** detrás de un balanceador de carga protege el Gateway, el servicio de Entradas y el clúster Redis; el **Circuit Breaker** protege las integraciones más propensas a fallar por causas externas (pasarela de pagos, proveedor de notificaciones, y las llamadas del Gateway hacia microservicios lentos), evitando que una falla externa se propague en cascada al resto del sistema.

8.3. Desplegabilidad – Media

Siguiendo el catálogo de tácticas (administrar el pipeline de despliegue, administrar el sistema desplegado) y patrones de reemplazo de servicios (Blue/Green, Rolling Upgrade, Canary Testing, A/B Testing) de Bass/Kazman visto en clase, la Tabla 3 identifica las tácticas y patrones aplicables a los escenarios de despliegue más relevantes del sistema.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Hotfix al servicio de validación de QR mientras hay ingreso masivo activo	Hay que corregir un defecto sin interrumpir las validaciones en curso ni arriesgar inconsistencia de versión durante el pico de tráfico.	<i>Scripts de despliegue</i> automatizados; inmediato si la nueva versión falla su <i>health-check</i> .	Blue/Green (instancias nuevas junto a las antiguas, corte de tráfico solo cuando la versión nueva responde sana; se acepta el costo de $2 \times N$ instancias porque la prioridad en un evento en vivo es cero inconsistencia de versión).

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Actualización rutinaria de un microservicio fuera de horario de evento	Se libera una nueva versión sin urgencia, con más tolerancia a un breve periodo de coexistencia de versiones.	Scripts de despliegue; <i>scaled rollout</i> por instancia.	Rolling Upgrade (más económico, solo N+1 instancias; se prefiere sobre Blue/Green porque aquí sí es aceptable una inconsistencia temporal de versión, a cambio de menor costo).
Cambio riesgoso en la integración con la pasarela de pagos	Un cambio en cómo se procesan los cobros no debe exponerse a todos los clientes de una vez, por si introduce un defecto financiero.	<i>Scaled rollout</i> a un subconjunto de usuarios; <i>feature toggle</i> para desactivarlo de inmediato si falla.	Canary Testing (se prefiere sobre A/B Testing porque el objetivo es detectar fallas con bajo riesgo antes de liberar a todos, no comparar métricas de negocio entre dos versiones).

Tabla 3: Tácticas y patrones de desplegabilidad aplicados a los escenarios de despliegue del sistema.

Se eligió **Blue/Green** para el escenario de mayor riesgo (evento en vivo) y **Rolling Upgrade** para el de menor riesgo (fuera de horario) en vez de aplicar un único patrón a todo el sistema, porque el propio catálogo de Bass/Kazman los presenta como alternativas con el mismo objetivo (reemplazo total de servicio) pero distinto trade-off costo/riesgo — no hay evidencia propia (PoC) todavía que mida ese trade-off en HEXACORE; queda como

uno de los PoCs recomendados en *Proyecto/Documentation/Work/GuiaEntrega1.md*.

8.4. Integrabilidad – Media

Siguiendo el catálogo de tácticas (limitar dependencias, adaptar, coordinar) y patrones (Wrapper/Adapter, SOA, descubrimiento dinámico) de Bass/Kazman visto en clase, la Tabla 4 identifica las tácticas y patrones aplicables a las integraciones externas del sistema.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Integración con la Pasarela de Pagos externa	El sistema debe coordinar/reembolsar sin acoplar cada microservicio al protocolo específico del proveedor de pagos, y poder cambiar de proveedor sin reescribir la lógica de negocio.	Adaptar interfaces; a estándares (REST/HTTPS); abstraer servicios comunes (interfaz única <i>Procesador de Pagos</i>).	Wrapper/Adapter (un único componente traduce hacia el protocolo del proveedor externo; se prefiere sobre exponer el SDK del proveedor directamente en cada microservicio, que acoplaría fuertemente el dominio a un proveedor concreto).

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Integración con el Proveedor de Notificaciones (push/correo/SMS) (push/correo/SMS) que finalmente se use.	El sistema debe enviar notificaciones sin que la lógica de negocio conozca el canal específico (push, correo o SMS) que finalmente se use.	Abstraer servicios comunes (interfaz única de notificación); usar intermediarios (la cola de mensajes ya decidida en ADR-04 desacopla el envío del flujo síncrono).	Adapter sobre la cola de mensajes como intermediario asíncrono (se prefiere sobre llamar directamente al SDK de cada canal, que dificultaría añadir un canal nuevo — ej. WhatsApp — sin tocar cada microservicio emisor).
Futuras integraciones con sistemas de la organización (Contabilidad, CRM, BI — ver <i>System Landscape</i> en <i>C4Diagrams.tex</i>)	Otros sistemas de la organización deberán poder consumir datos de HEXACORE (ventas, ocupación) sin acceder directamente a las bases de datos de cada microservicio.	Adaptar interfaces (API Gateway como punto único de salida); adhesión a estándares (REST/JSON).	API Gateway reutilizado también como puerta de integración saliente (se prefiere sobre exponer cada base de datos de microservicio directamente, que rompería el aislamiento por dominio del ADR-01).

Tabla 4: Tácticas y patrones de integrabilidad aplicados a las integraciones externas del sistema.

8.5. Comprobabilidad (*Testability*) – Media

Siguiendo el catálogo de tácticas (controlar y observar el estado del sistema, limitar complejidad) y patrones (inyección de dependencias, patrón estrategia, filtros de intercepción) de Bass/Kazman visto en clase, la Tabla 5 identifica las tácticas y patrones aplicables a los flujos más difíciles de probar del sistema.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Probar el flujo de reventa (CU-006) sin depender de la Pasarela de Pagos real	Las pruebas automatizadas del Servicio de Entradas no deben fallar ni cobrar dinero real por una indisponibilidad del proveedor externo.	Sandboxing / abstracción de fuentes de datos (mock del <i>Procesador de Pagos</i>); interfaces especializadas para inspeccionar el estado del bloqueo de Redis en cada prueba.	Inyección de dependencias (el <i>Procesador de Pagos</i> se recibe como interfaz inyectable, no se instancia internamente — se prefiere sobre acoplar el componente al SDK real, que haría imposible probarlo sin red).
Reproducir un defecto de condición de carrera en el bloqueo de concurrencia de Redis	Un error reportado en producción (dos compradores casi simultáneos) debe poder reproducirse de forma determinista en un ambiente de pruebas.	Reducir no-determinismo (controlar explícitamente el orden de llegada de las peticiones concurrentes en la prueba); <i>record/playback</i> del estado de Redis antes/después.	Sin patrón arquitectónico dedicado — se resuelve con disciplina de pruebas (harness de concurrencia controlada) más que con un componente nuevo.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Aislar las pruebas de un microservicio sin levantar todo el sistema	Un integrante debe poder correr las pruebas de su microservicio (ej. Par-queaderos) sin tener las bases de datos y colas de los demás dominios corriendo.	Localizar el almacenamiento del estado del sistema (cada microservicio ya lo hace, por ADR-01); interfaces especializadas (endpoints de salud/estado para pruebas de integración).	Filtros de intercepción en las pruebas de integración, para registrar y verificar las peticiones salientes hacia otros servicios sin necesitar que respondan de verdad.

Tabla 5: Tácticas y patrones de comprobabilidad aplicados a los flujos más difíciles de probar del sistema.

8.6. Seguridad física (*Safety*) – Baja

A diferencia de los demás atributos de esta sección, **Safety no es un atributo prioritario para HEXACORE**: el catálogo de Bass/Kazman para Safety está pensado sobre todo para sistemas de control físico/embebidos (aviónica, industria, salud) y HEXACORE es un sistema de información, no de control de hardware crítico — se documenta aquí por completitud, como exige el rango de clases 6-14, y porque el sistema sí participa de forma indirecta en la seguridad física de los asistentes a través del control de aforo y la coordinación de evacuaciones (CU-010).

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Se excede el aforo máximo de una zona (riesgo de aglomeración)	El sistema debe impedir que se registren más ingresos en una zona que ya alcanzó su aforo máximo, sin importar que la entrada ya esté pagada.	Barrera/Interlock (bloquear el registro de ingreso, no solo advertirlo); monitoreo de condiciones (aforo actual vs. máximo configurado en tiempo real).	Separación de Safety — el control de aforo se trata como una verificación independiente y con prioridad sobre el flujo comercial de validación de QR, no como una regla de negocio más del mismo componente.
Las instrucciones de evacuación mostradas al personal quedan desactualizadas (ruta bloqueada)	El sistema no debe sugerir una ruta de evacuación que ya se marcó como bloqueada por el supervisor.	Chequeo de sanidad (validar la ruta contra el estado actual antes de mostrarla); degradación elegante (mostrar una instrucción genérica de respaldo si no hay ruta válida).	Sin patrón dedicado — se apoya en el mismo Servicio de Eventos/Emergencias que ya coordina CU-010.

Tabla 6: Tácticas y patrones de seguridad física (Safety) aplicados al control de aforo y evacuación.

8.7. Seguridad – Alta

El API Gateway centraliza autenticación y autorización. Se propone control de acceso basado en roles:

Rol	Ejemplos de permisos
Cliente	Consultar eventos, comprar y gestionar sus entradas.
Empleado	Consultar turnos, registrar asistencia y reportar incidentes.
Supervisor	Supervisar personal, aprobar cambios y operar protocolos.
Organizador	Gestionar eventos, personal y operación.
Administrador	Administrar usuarios, roles y configuración.

Siguiendo el catálogo de tácticas (detectar, resistir, reaccionar y recuperarse de ataques) y patrones (interceptor de validación, sistema de prevención de intrusiones) de Bass/Kazman visto en clase, la Tabla 7 identifica las tácticas y patrones aplicables a los escenarios de seguridad más críticos del sistema.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Acceso no autorizado a datos de pago o credenciales	Un usuario sin sesión válida, o con permisos insuficientes, intenta acceder a datos de pago o credenciales de otro usuario.	Autenticar actores (tokens JWT); autorizar actores (RBAC, tabla anterior); cifrar datos (TLS en tránsito, cifrado en reposo para datos de pago); limitar exposición (principio de mínimo privilegio en el API Gateway).	Interceptor de validación en el API Gateway (se prefiere sobre validar permisos dentro de cada microservicio, que duplicaría la lógica de autorización en los seis dominios — ya decidido en ADR-02).

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Ataque de fuerza bruta contra el login	Un actor intenta muchas combinaciones de contraseña contra la misma cuenta en poco tiempo.	Detectar ataques (patrón de múltiples intentos fallidos); reaccionar (limitar/bloquear el login tras N intentos, ej. 5); informar al usuario legítimo del bloqueo.	Interceptor de validación (rate limiting) en el API Gateway, re-tras utilizando el mismo componente del escenario anterior en vez de introducir uno nuevo.
Un usuario niega haber realizado una compra, reventa o pago	Debe poder demostrarse, ante un reclamo, quién hizo qué operación y cuándo.	Auditar (registro de acciones con usuario, fecha y resultado — ya cubierto por ASR-07); no repudiación (el registro de auditoría queda firmado/inmutable, no editable retroactivamente).	Sin patrón arquitectónico — dedicado — se apoya en el mismo mecanismo de auditoría de ASR-07, reforzado con inmutabilidad del registro (ej. <i>append-only log</i>).

Tabla 7: Tácticas y patrones de seguridad aplicados a los escenarios críticos del sistema.

Se eligió centralizar autenticación/autorización/rate-limiting en el **API Gateway** (un solo interceptor de validación) en vez de replicar estas tácticas en cada microservicio, porque ya es el punto único de entrada decidido en ADR-02 — evita seis implementaciones distintas de lo mismo, al costo de que el Gateway concentra más responsabilidad crítica (mitigado con redundancia, ya cubierto en la sección de Disponibilidad).

8.8. Rendimiento – Alta

Siguiendo el catálogo de tácticas (controlar demanda de recursos, administrar recursos) y patrones (balanceador de carga, *throttling*, Map-Reduce) de Bass/Kazman visto en clase, la Tabla 8 identifica las tácticas y patrones aplicables a los escenarios de carga más exigentes del sistema.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Miles de solicitudes concurrentes al Servicio de Entradas durante la apertura de venta	La mayoría de las solicitudes de consulta/compra deben resolverse en tiempos cortos y estables (ASR-04), sin que el pico degrade a los demás microservicios.	Administrar peticiones de trabajo (SLA de tiempo de respuesta); introducir concurrencia (múltiples nodos, réplicas); aumentar recursos (autoescalado).	Balanceador de Carga + escalado horizontal del microservicio de Entradas (mismo patrón usado para Disponibilidad — aquí se elige por reducir latencia, no solo por tolerancia a fallos).
Validación de QR en el ingreso al recinto	Cada validación debe resolverse en menos de 500 ms (RNF-01) aunque haya miles de asistentes ingresando en pocos minutos.	Aumentar eficiencia (índices en la tabla de entradas); mantener múltiples copias de datos (caché de entradas válidas del evento en curso).	Caching con Redis (se prefiere sobre consultar siempre la base de datos relacional, que sería más lenta bajo esa frecuencia de lectura — reutiliza la infraestructura Redis ya decidida en ADR-03).

Escenario	Estímulo / Respuesta esperada	Tácticas	apli-	Patrón
Generación de reportes agregados pesados (ventas, ocupación) en el Servicio de Administración	Un reporte que cruza datos de varios eventos no debe bloquear ni ralentizar las operaciones transaccionales del mismo servicio.	Introducir concurrencia (procesamiento paralelo/asíncrono); limitar tamaño de colas para no saturar el procesamiento de reportes.	con- par-	Map-Reduce sobre los datos ya replicados a MongoDB (polyglot persistence, ADR-01) — se prefiere sobre calcular el reporte con consultas síncronas directas a PostgreSQL, que competiría por recursos con las transacciones de venta en curso.

Tabla 8: Tácticas y patrones de rendimiento aplicados a los escenarios de carga del sistema.

8.9. Tiempo real – Media-Alta

Aforo, entradas, parqueaderos, personal y emergencias requieren actualizaciones cercanas al tiempo real. Redis, colas y mecanismos de comunicación en tiempo real, como WebSockets, pueden utilizarse para estos escenarios.

8.10. Escalabilidad – Media

Los servicios se pueden escalar individualmente. Durante una preventa puede crecer el servicio de Entradas, mientras que durante el ingreso puede aumentar principalmente la capacidad de validación.

8.11. Trazabilidad – Media

Se deben registrar operaciones como transferencias, compras, reembolsos, cambios de turno, accesos, modificaciones administrativas y emergencias.

8.12. Usabilidad – Baja-Media

A diferencia de los demás atributos de esta sección, la Clase 14 no cubrió tácticas/patrones de Bass/Kazman para Usabilidad, sino el **Five Planes Framework** de Jesse James Garrett — se aplica ese marco aquí, no una tabla de tácticas, para mantener la coherencia con lo visto en clase.

- **Estrategia:** el objetivo de negocio es maximizar las compras/reventas completadas sin fricción; los usuarios finales van desde clientes frecuentes de eventos hasta compradores ocasionales con distinto nivel de familiaridad tecnológica — la interfaz no puede asumir experiencia previa.
- **Alcance:** funcionalidad (comprar/revender entradas, reservar parqueadero, pedir comida, ver el estado de todo eso) y contenido (fecha, lugar, precio, disponibilidad y aforo de cada evento) ya delimitados por los CU-001 a CU-025.
- **Estructura:** arquitectura de información jerárquica (Inicio → Evento → Entradas del evento), con un modelo conceptual explícito de "boletera digital" (análogo a apps de boletería como TuBoleta Pass) — ya adoptado como referencia de diseño en `EntradasScreen` de la app móvil.
- **Esqueleto:** navegación por pestañas inferiores (Inicio/Parqueadero/Pedidos para Cliente; Turnos/Asistencia/función operativa/Incidentes/Emergencia para Personal) más un menú lateral común para Perfil/Ajustes — ya implementado igual en `app-movil` y `app-ios`.
- **Superficie:** pendiente de definir una identidad visual de marca propia; hoy las apps usan la paleta por defecto de Material 3 (Android) y del sistema (iOS) — riesgo menor de inconsistencia visual entre plataformas, marcado como pendiente en el código (`ui/theme/Theme.kt`).

Cada tipo de usuario posee además una interfaz especializada (cliente, personal, administración), evitando presentarle a cada uno funciones que no necesita — esto es, en términos del Five Planes, una decisión del plano de Alcance (qué funcionalidad expone cada interfaz) que se sostiene en la separación de contenedores ya decidida en la vista de contenedores (Sección ?? del SAD).

8.13. Mantenibilidad – Baja-Media

Siguiendo el catálogo de tácticas (incrementar cohesión, reducir acoplamiento, postergar enlace) y patrones (Layered/N-Tier, Service-based, Event-driven, Microkernel) de Bass/Kazman visto en clase, la Tabla 9 identifica las tácticas y patrones aplicables a los escenarios de cambio más probables del sistema.

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Cambiar una regla de negocio de Parqueaderos sin afectar Entradas (ASR-09)	Un desarrollador modifica y redespiega el Servicio de Parqueaderos sin necesitar tocar ni redesplegar el Servicio de Entradas.	Encapsular (interfaz del servicio); restringir dependencias (cada microservicio solo se comunica vía su API, nunca accede a la BD de otro).	Service-based (microservicios, ya decidido en ADR-01) — se reafirma aquí desde el ángulo de Modificabilidad, no solo de Escalabilidad.
Agregar un nuevo método de pago sin modificar cada microservicio que cobra	Los seis microservicios que hoy cobran contra la Pasarela de Pagos no deberían cambiar su código al agregar, por ejemplo, un nuevo método de pago digital.	Abstraer servicios comunes (interfaz única <i>Procesador de Pagos</i>); usar intermediarios.	Adapter/Facade sobre la Pasarela de Pagos (mismo componente reutilizado desde la sección de Integrabilidad — la abstracción resuelve ambos atributos a la vez).

Escenario	Estímulo / Respuesta esperada	Tácticas aplicadas	Patrón
Cambiar el motor de cola de mensajes en el futuro (de RabbitMQ a Kafka, o viceversa)	Los productores/consumidores de eventos (generación de QR, notificaciones, auditoría) no deberían reescribirse si cambia el motor de mensajería subyacente.	Usar intermediarios (publicar/consumir contra una interfaz propia, no contra el SDK del motor directamente); postergar enlace (configuración del motor concreto en tiempo de despliegue, no de compilación).	Event-driven Architecture , con una capa de abstracción propia sobre el motor concreto — se prefiere sobre acoplar cada microservicio al SDK específico de RabbitMQ o Kafka, que haría costoso el cambio de motor decidido en ADR-04.

Tabla 9: Tácticas y patrones de mantenibilidad aplicados a los escenarios de cambio más probables del sistema.

8.14. Portabilidad – Baja-Media

Los servicios del backend son consumidos mediante APIs, por lo que pueden ser utilizados desde el portal web del cliente, portal administrativo y aplicaciones móviles sin duplicar la lógica de negocio.

9. Caso complejo: Gestión de Evacuación ante Emergencias

Este caso de uso se propone como un proceso complejo porque integra varios dominios y requiere disponibilidad, seguridad, rendimiento, tiempo real y trazabilidad.

9.1. Objetivo

Permitir que un usuario autorizado active, coordine, supervise y finalice un protocolo de evacuación utilizando información de zonas, aforo, salidas, rutas y personal operativo.

9.2. Actores

- Supervisor de emergencia.
- Organizador.
- Personal de seguridad.
- Personal médico.
- Asistentes.
- Servicio de notificaciones.

9.3. Flujo principal

1. El supervisor inicia sesión en el módulo de emergencias.
2. Selecciona “Activar evacuación”.
3. El sistema solicita el tipo y ubicación de la emergencia.
4. El supervisor registra la información.
5. El sistema identifica las zonas potencialmente afectadas.
6. Consulta el aforo actual.
7. Identifica las salidas disponibles.
8. Determina las rutas configuradas.
9. Presenta las rutas y zonas recomendadas.
10. El supervisor confirma el protocolo.
11. El sistema notifica al personal operativo.
12. El sistema envía instrucciones a los asistentes.
13. El sistema monitorea las zonas.
14. El supervisor consulta el avance y atiende alertas.
15. El supervisor confirma la finalización y el sistema registra la emergencia.

9.4. Flujos alternativos

Ruta bloqueada: si una salida deja de estar disponible, el sistema la retira de las rutas recomendadas, busca una alternativa y actualiza las instrucciones.

Zona congestionada: el sistema genera una alerta y el supervisor puede seleccionar una ruta alternativa.

Evacuación parcial: solamente se notifican y controlan las zonas afectadas.

Cambio del nivel de emergencia: el sistema vuelve a evaluar zonas y rutas cuando aumenta la gravedad.

9.5. Excepciones

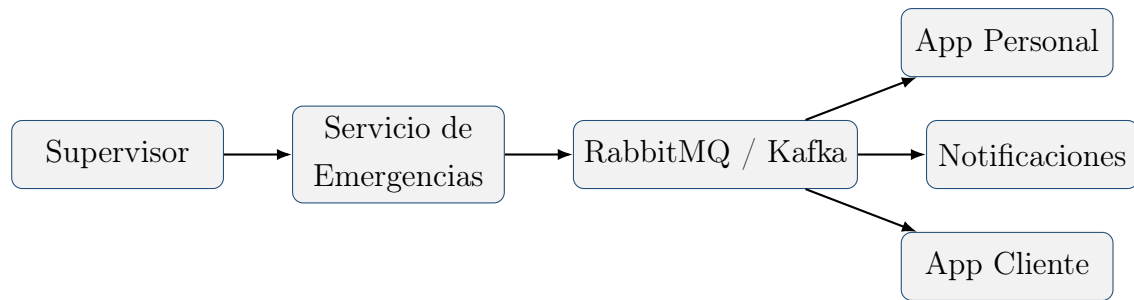
- No existen rutas disponibles.
- Se pierde comunicación con el monitoreo.
- Falla el servicio de notificaciones.
- El usuario no tiene permisos.
- Se pierde temporalmente la conexión de una aplicación.

9.6. Atributos de calidad del caso

- **Disponibilidad:** el servicio debe permanecer operativo durante la emergencia.
- **Rendimiento:** las alertas deben procesarse rápidamente.
- **Tiempo real:** el estado de las zonas debe actualizarse con baja latencia.
- **Seguridad:** solo usuarios autorizados pueden activar o modificar el protocolo.
- **Trazabilidad:** las acciones deben quedar registradas.

10. Comunicación durante una emergencia

Un escenario representativo es un concierto en el que se detecta un incendio en una zona. El supervisor activa el protocolo desde el portal administrativo. El servicio de emergencias consulta aforo, zonas y rutas y publica eventos para notificar al personal y a los asistentes.



11. Relación entre interfaces y microservicios

Servicio	Interfaces	Responsabilidad
Entradas	Web Cliente, App Cliente	Compra, disponibilidad, validación y gestión.
Mercado Secundario	Web Cliente	Publicación, compra y transferencia.
Personal	Web Administrativo, App Personal	Empleados, turnos, asignaciones y asistencia.
Eventos	Web Administrativo, Web Cliente, Apps	Gestión y consulta de eventos.
Parqueaderos	Web Cliente, Web Administrativo, Apps	Reservas, espacios, cobros y ocupación.
Emergencias	Web Administrativo, App Personal, App Cliente	Activación, coordinación, alertas y seguimiento.

12. Resumen de decisiones arquitectónicas

Decisión	Problema	Atributos
Microservicios	Separación de dominios y escalamiento	Escalabilidad, mantenibilidad, rendimiento
API Gateway	Punto central de acceso y seguridad	Seguridad, mantenibilidad
Balanceador	Distribución de solicitudes	Disponibilidad, rendimiento
Redis	Acceso rápido y bloqueo temporal	Consistencia, tiempo real, rendimiento
RabbitMQ/Kafka	Procesamiento asíncrono	Rendimiento, escalabilidad
BD por servicio	Evitar acoplamiento de datos	Mantenibilidad, escalabilidad
Notificaciones	Comunicación desacoplada	Rendimiento, tiempo real
Pasarela de pagos	Transacciones externas	Seguridad, consistencia
Interfaces especializadas	Adaptación al usuario	Usabilidad, portabilidad

Tabla 11: Resumen de decisiones arquitectónicas.

13. Conclusión

La arquitectura propuesta conserva las decisiones fundamentales de la primera versión, pero las adapta a una definición más concreta del sistema: un portal web para clientes, un portal web administrativo y operativo, una aplicación móvil para clientes y una aplicación móvil para el personal.

La separación de interfaces permite que cada tipo de usuario tenga las funcionalidades que necesita, mientras que el backend mantiene la lógica de negocio centralizada en servicios independientes.

Los atributos de calidad prioritarios siguen siendo consistencia, disponibilidad, seguridad y rendimiento. Estos se soportan mediante microservicios, API Gateway, balanceador de carga, Redis, colas de mensajes, bases de datos independientes y servicios externos de pago y notificaciones.

La incorporación de aplicaciones móviles aumenta la importancia de tiempo real, usabilidad y portabilidad. Asimismo, la gestión de evacuación ante emergencias constituye un caso complejo adecuado para demostrar cómo las decisiones arquitectónicas responden a atributos de calidad concretos.